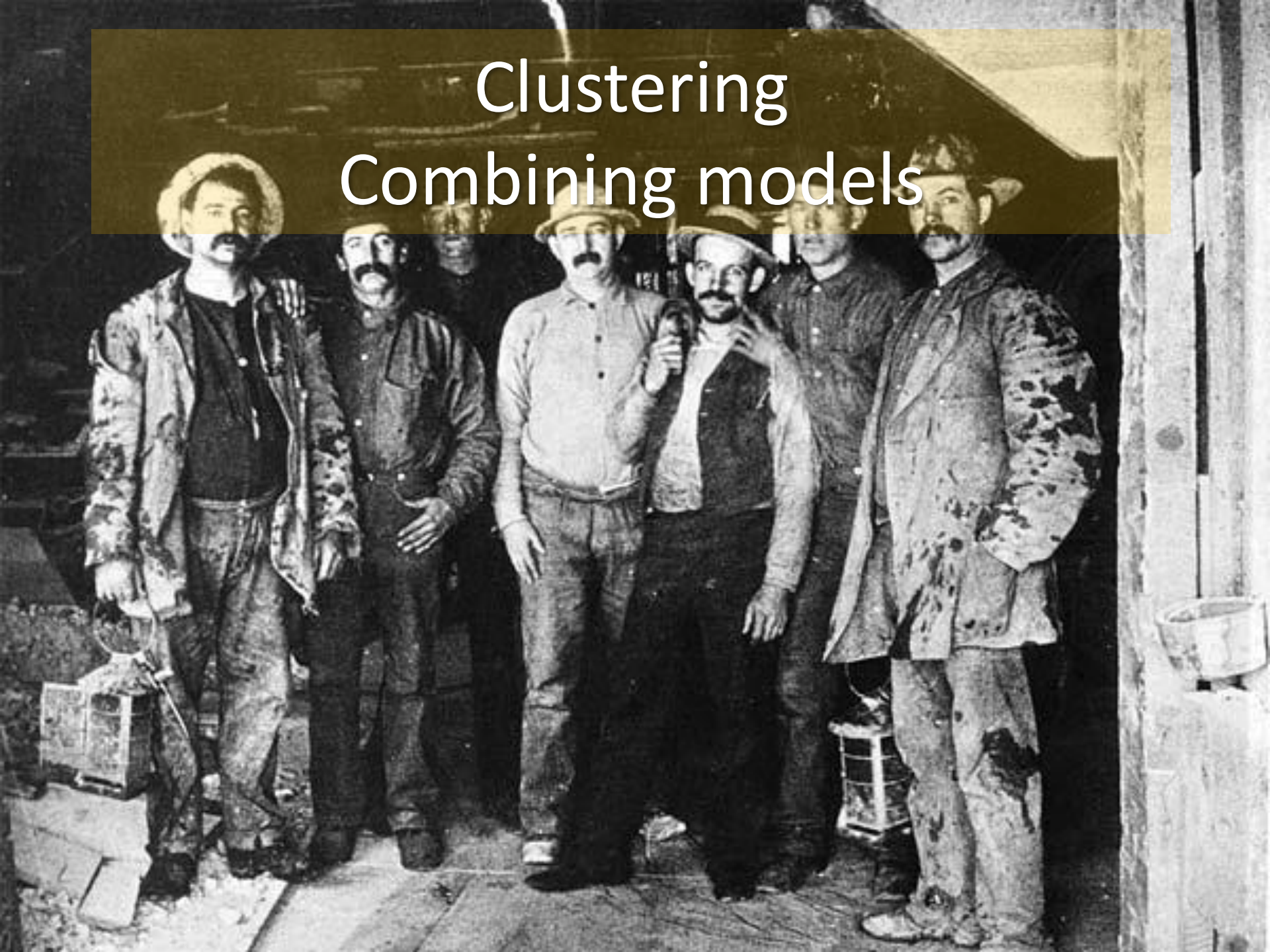


Clustering Combining models



Clustering (1/2)

- What is clustering?

*The task of grouping a set of objects in such a way that objects in the same group (called a cluster) are more **similar** (in some sense or another) to **each other than to those in other groups** (clusters)*

- A form of *unsupervised* learning
 - no (labels for) target variable

Clustering (2/2)

- What do we need?
 - Distance metric (how far away are instances in our dataset)
 - Clustering algorithm

Distance metrics (1)

- Individual distance metrics (between an instance x_i and x_j over all attributes p):

- Euclidean distance

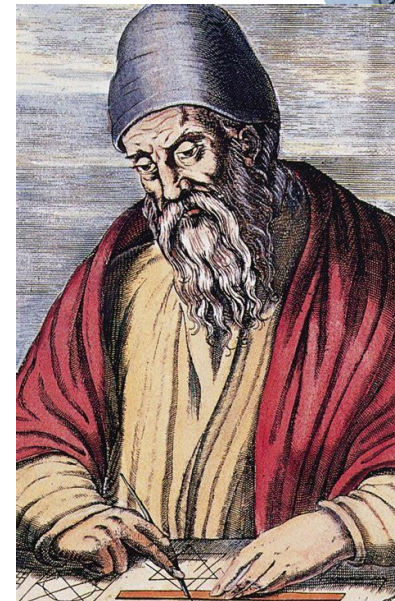
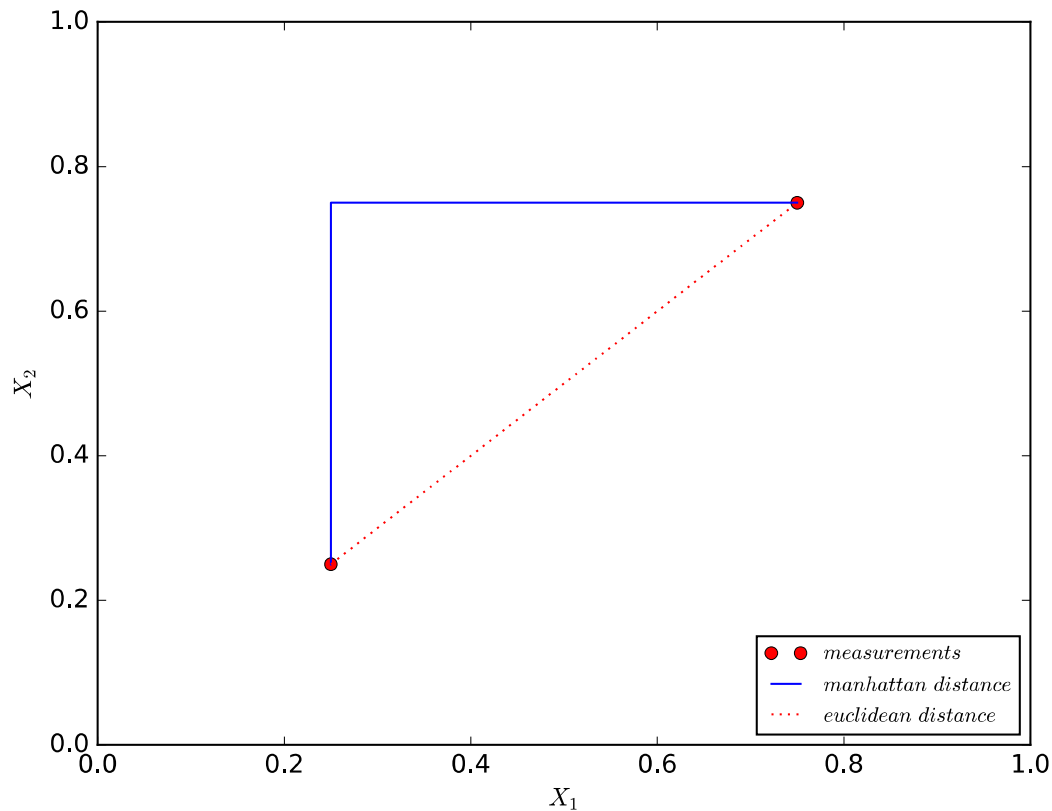
$$euclidian_distance(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_i^k - x_j^k)^2}$$

- Manhattan distance

$$manhattan_distance(x_i, x_j) = \sum_{k=1}^p |x_i^k - x_j^k|$$

Distance metrics (2)

- The difference:



Distance metrics (3)

- Generalized form:

$$\text{minkowski_distance}(q, x_i, x_j) = \left(\sum_{k=1}^p |x_i^k - x_j^k|^q \right)^{\frac{1}{q}}$$

- Important to consider: scaling of data
- These all assume numeric values, what if we have other types?
 - We can use Gower's similarity

Distance metrics (4)

- Gower's similarity for different types of features:

- Dichotomous attributes (present or not)

$$s(x_i^k, x_j^k) = \begin{cases} 1 & \text{when } x_i^k \text{ and } x_j^k \text{ are both present} \\ 0 & \text{otherwise} \end{cases}$$

- Categorical attributes:

$$s(x_i^k, x_j^k) = \begin{cases} 1 & \text{when } x_i^k = x_j^k \\ 0 & \text{otherwise} \end{cases}$$

- Numerical attributes:

$$s(x_i^k, x_j^k) = 1 - \frac{|x_i^k - x_j^k|}{R_k} \text{ where } R_k \text{ is the range of } k$$

already
normalized!

Distance metrics (5)

- And we compute the final similarity as:

$$\text{gowers_similarity}(x_i, x_j) = \frac{\sum_{k=1}^p s(x_i^k, x_j^k)}{\sum_{k=1}^p \delta(x_i^k, x_j^k)}$$

$$\delta(x_i^k, x_j^k) = \begin{cases} 1 & \text{when } x_i^k \text{ and } x_j^k \text{ can be compared (i.e. both have values)} \\ 0 & \text{otherwise} \end{cases}$$

Clustering approaches

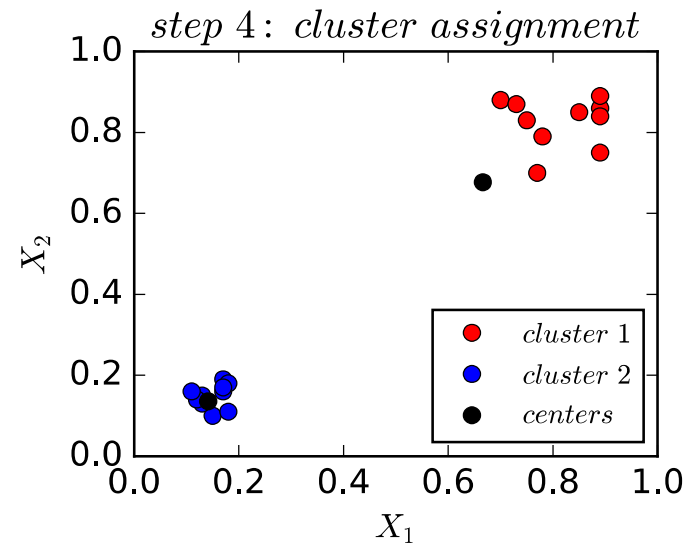
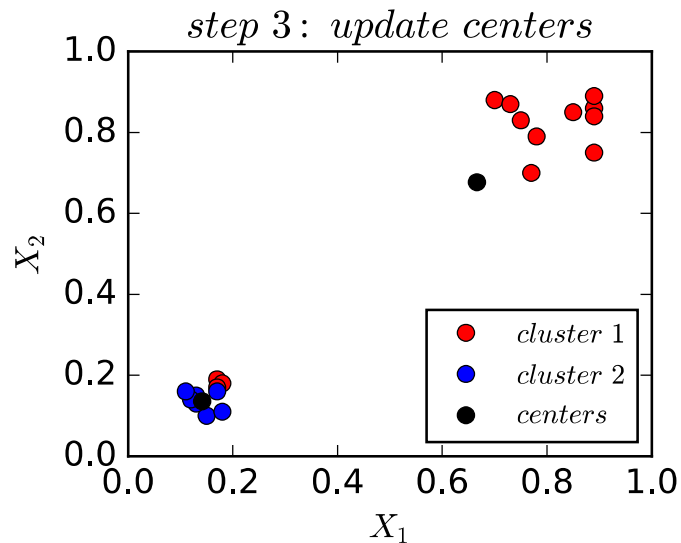
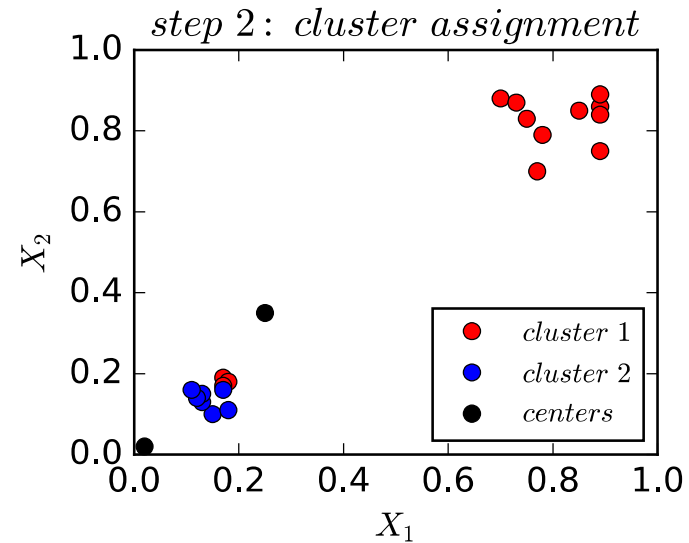
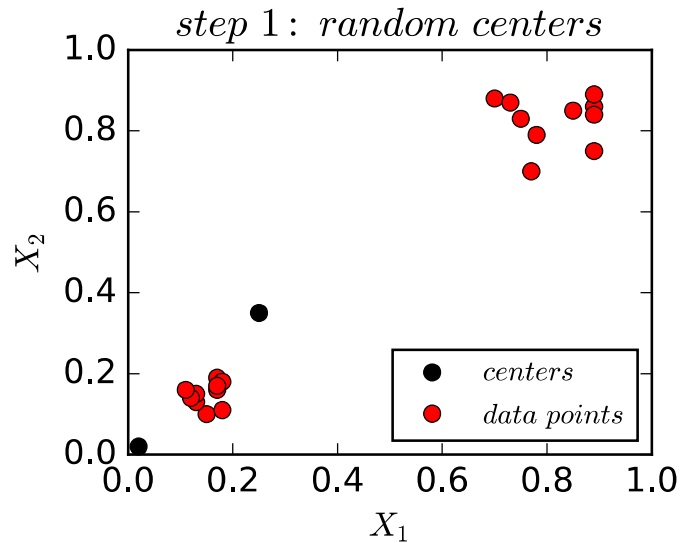
- Ok, we know how to define a distance, now what clustering approaches do we have?
- Three examples:
 - k-means
 - hierarchical clustering

k-means clustering (1)

Algorithm 4: k-means clustering

```
for  $i = 1, \dots, k$  do
    | centers[k] = random point in the clustering space
end
prev_centers = []
while prev_centers  $\neq$  centers do
    | prev_centers = centers
    | cluster_assignment = []
    | for  $i = 1, \dots, N$  do
    | | cluster_assignment[i] =  $\operatorname{argmin}_{j=1, \dots, k} \text{distance}(x_i, \text{centers}[j])$ 
    | end
    | for  $j = 1, \dots, k$  do
    | | centers[j] =  $\frac{\sum_{i \in \{i | \text{cluster\_assignment}[i] = j\}} x_i}{|\{i | \text{cluster\_assignment}[i] = j\}|}$ 
    | end
end
end
```

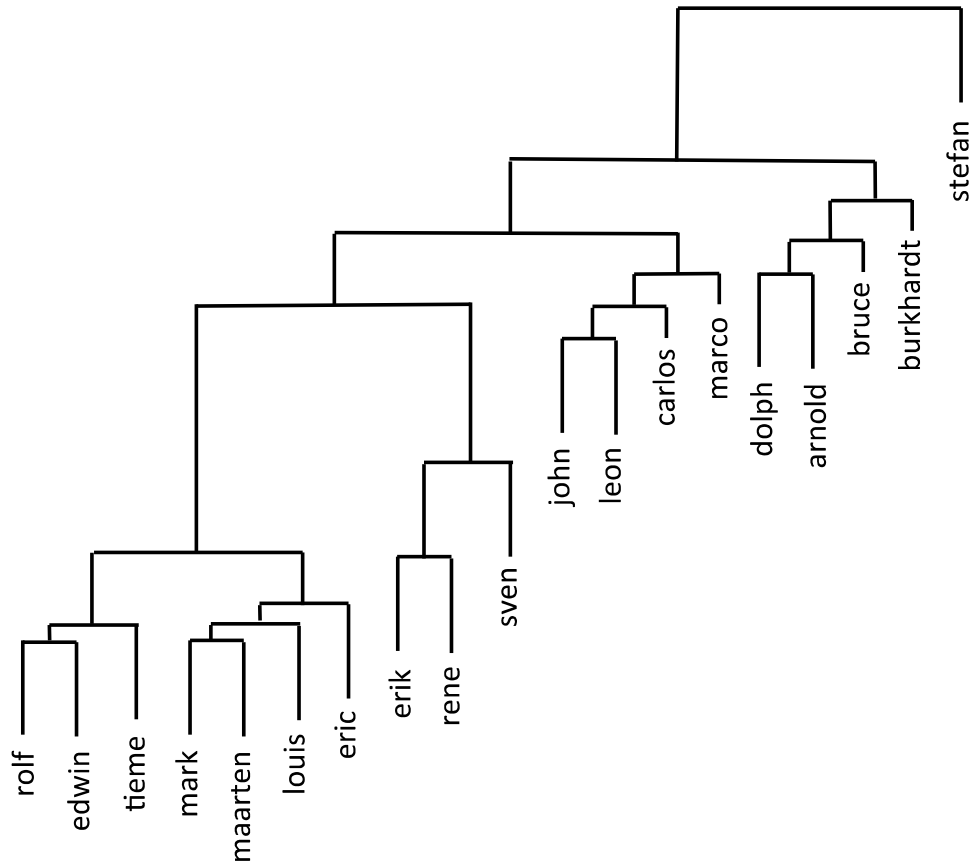
k-means clustering (2)



Hierarchical clustering (1)

- Can also take a more iterative approach:
 - Divisive clustering
 - Start with one cluster, make a split in each step
 - Agglomerative clustering
 - Start with one cluster per instance and merge
 - Can be visualized using a dendrogram

Hierarchical clustering (2)



Agglomerative clustering (1)

- Start with one cluster per instance and merge
- We need to know which clusters to merge
- Different criteria:

- Single linkage

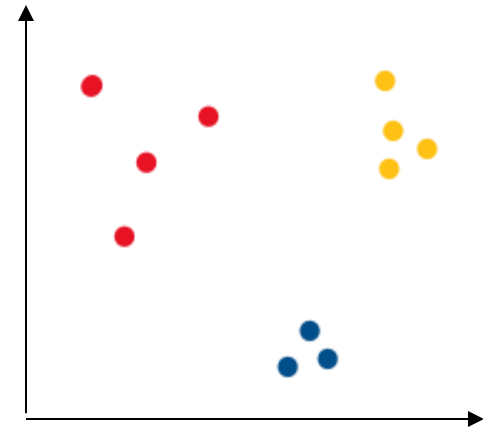
$$d_{SL}(C_k, C_l) = \min_{x_i \in C_k, x_j \in C_l} distance(x_i, x_j)$$

- Complete linkage

$$d_{CL}(C_k, C_l) = \max_{x_i \in C_k, x_j \in C_l} distance(x_i, x_j)$$

- Group average

$$d_{GA}(C_k, C_l) = \frac{\sum_{x_i \in C_k} \sum_{x_j \in C_l} distance(x_i, x_j)}{|C_k| \cdot |C_l|}$$



Agglomerative clustering (2)

- We need to know which clusters to merge
- Different criteria:
 - Ward's criterion (minimize standard deviation):

$$d_{Ward}(C_k, C_l) = \sum_{x_i \in C_k \cup C_l} \|x_i - m_{C_k \cup C_l}\|^2 - \sum_{x_j \in C_k} \|x_j - m_{C_k}\|^2 - \sum_{x_n \in C_l} \|x_n - m_{C_l}\|^2$$

Agglomerative clustering (3)

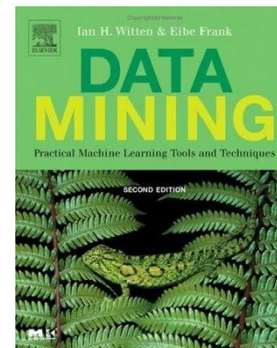
Algorithm 6: Agglomerative clustering

```
clusters = {}  
for  $x_i \in \mathbf{X}$  do  
    | clusters = clusters +  $\{x_i\}$   
end  
while True do  
    |  $C_k, C_l = \operatorname{argmin}_{C_k \in \text{clusters}, C_l \neq C_k \in \text{clusters}} d(C_k, C_l)$   
    | if  $d(C_k, C_l) > th$  then  
    |     | return clusters  
    | else  
    |     | clusters = clusters  $\setminus \{C_k\}$   
    |     | clusters = clusters  $\setminus \{C_l\}$   
    |     | clusters = clusters +  $\{C_k + C_l\}$   
    |  
end
```

Ensembles

- Bagging
- Boosting
- Stacking
- Error correcting codes

Chapter 7.4



Ensembles of classifiers

- Main idea:
 - instead of building a single classifier build several classifiers and combine their outputs (e.g., by majority voting, weighted voting, etc.)
- Such a combination usually works better than single models.



Motivation to you

- DM competitions are often won by ensemble methods
- Often used in industry
- The problem is though that such combinations are not easy to interpret, and thus it's hard to say when that combination will work on other problems

Bagging

- **Given**
 - a training set T and a learning method (e.g., ID3, Backprop, or Naive Bayes,...)
- Create, say, 100 versions of T by resampling it (with replacement):
$$T_1, \dots, T_{100}$$
- For each T_i build a model (classifier / regression)
- Return an “ensemble” of classifiers, *and then...*



...the committee votes

- Classification: majority vote
- Numeric prediction: average output

Variants

- Randomise (*to increase diversity*)
 - Instead of resampling data, randomly perturbate model parameters (Naïve Bayes, Neural Net)
 - E.g., build models using different random seeds (*if applicable*)
 - When running greedy search (decision tree), instead of selecting the best option, select at random one from the top k best options
 - E.g., not *the best* but *one of the best* attributes to split on
 - Etc.
- Instead of voting, average the probability values for classes (*if probabilities are available*)
- Can be extended to deal with misclassification costs

Popular Variant: Random Forest

- Basis:
 - Ho, T. K. (1995, August), Random decision forests
 - Breiman (2001), Random Forest
- We start with the basic bagging
 - We select a random sample of our training data
- Per sample we generate a decision tree as we have seen before
- We also sample *features* to split our tree on

Random Forest

Algorithm 15.1 *Random Forest for Regression or Classification.*

1. For $b = 1$ to B :
 - (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
 - (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
 - i. Select m variables at random from the p variables.
 - ii. Pick the best variable/split-point among the m .
 - iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x :

Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$.

Classification: Let $\hat{C}_b(x)$ be the class prediction of the b th random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \text{majority vote } \{\hat{C}_b(x)\}_1^B$.

How do we set the parameters?

- There are some recommendations.....
- Classification:
 - Set subset of variables m to $\sqrt{p}$
 - Set the minimum node size to one (!)
- Regression:
 - Set subset of variables m to $\frac{p}{3}$
 - Set the minimum node size to five

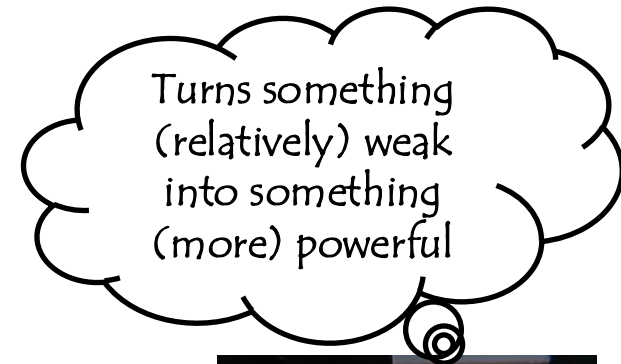
Bagging & expected error

- **Error** = **bias** + **variance**
- **Bias** – limitation of the learning method
 - Even infinite number of training instances would not be enough to achieve lower error than this
 - Zero bias: total (over)fitting
- **Variance** – limitation of the training data
 - Not representing the population well
 - Zero variance: when you always predict the same
- Bagging reduces **variance**
- Makes sense when:
 - the “basic learner” is sensitive to small changes in data
 - the “basic learner” overfits the data (e.g. a full tree)
 - the data is scarce

Boosting = learning from previous errors

- It's like bagging

- Voting (*class.*) or averaging (*num. pred.*)
- Same kinds of models built (*e.g., all are NB*)



- Yet it's not

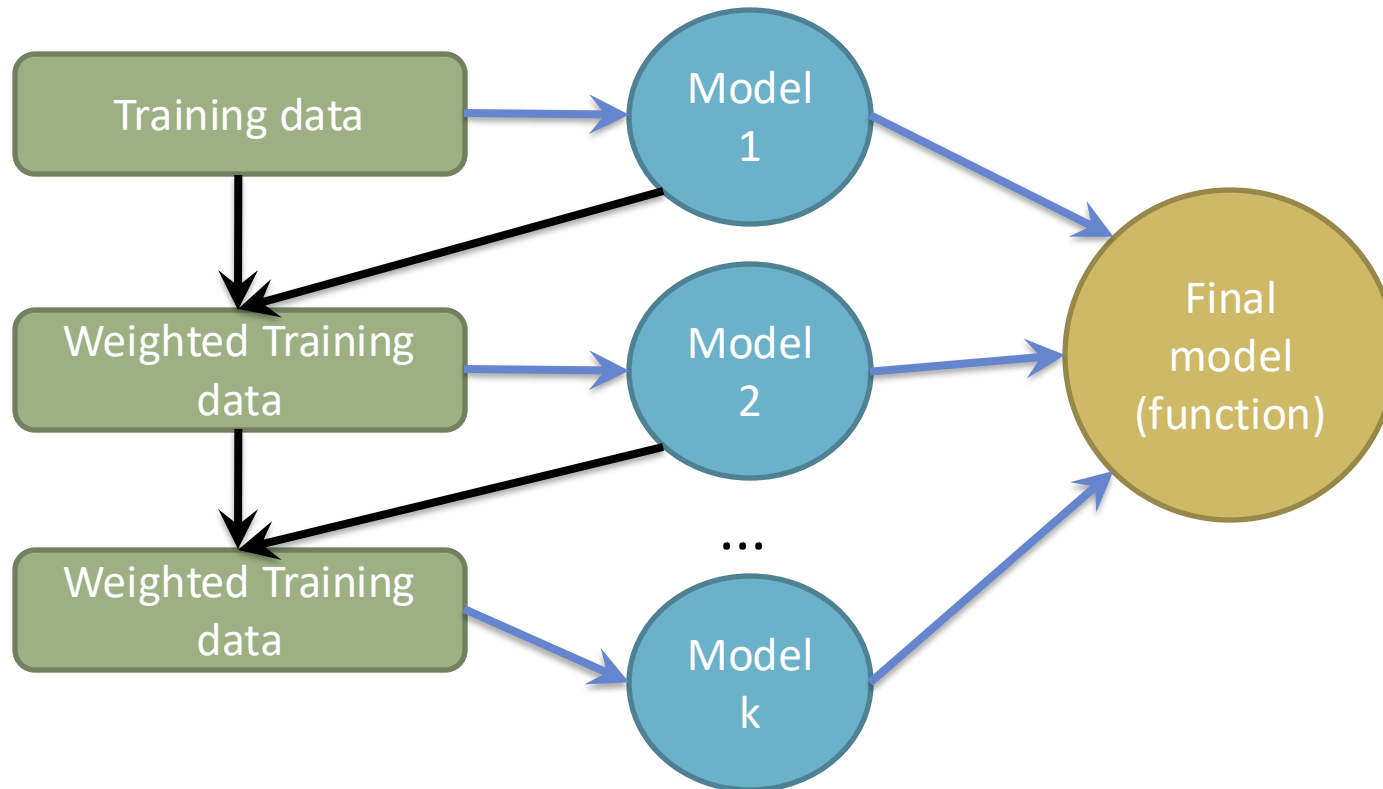
- Boosting is iterative (*one model at a time*)
- A newly added model should focus on parts where the previous ones make mistakes (*there will be 'experts'*)
- Reweighting instead of resampling (*based on expertise*)



Boosting, in other words

- Build a sequence of classifiers $C_1, C_2, \dots, C_k$, as follows:
 - C_1 is trained on the original data,
 - C_2 “pays more attention” to cases misclassified by C_1 ,
 - C_3 “pays more attention” to cases misclassified by C_1 and C_2 ,
 - etc.
- “pays more attention” = assigns higher weights
- In other words, boosting develops a number of ‘experts’ that specialize in different regions of the data

Boosting



Freund, Schapire, 96: Boosting

- $C_1, C_2, \dots, C_k$ are constructed as follows:
- Let C_1 be a “normal” classifier constructed on T
- For $i=2:k$
 - Assign to every case in T (*the dataset*) a **weight** that is proportional to the error made on this case by the current ensemble
 - Build next classifier C_i on **weighted data**; cases with higher **weights** are more important

Breiman, 96: Arcing

- $C_1, C_2, \dots, C_k$ are constructed as follows:
 - Let C_1 be a “normal” classifier constructed on T
 - For $i=2:k$
 - assign to every case in T a weight that is proportional to the number of classifiers that misclassify it
 - resample the data using these weights (T_i)
 - build the classifier C_i on the resampled data

Freund & Schapire, 96: AdaBoost

(Ada = adaptive)

- **Use:** Some techniques can directly work with weighted data (*e.g., neural network: weighted error function*).
- AdaBoost builds models that minimize the weighted error:

$$\sum w_i (\text{target}_i - \text{predicted}_i)^2$$

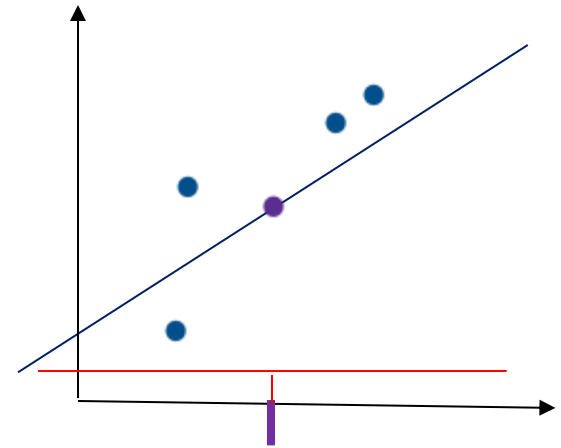
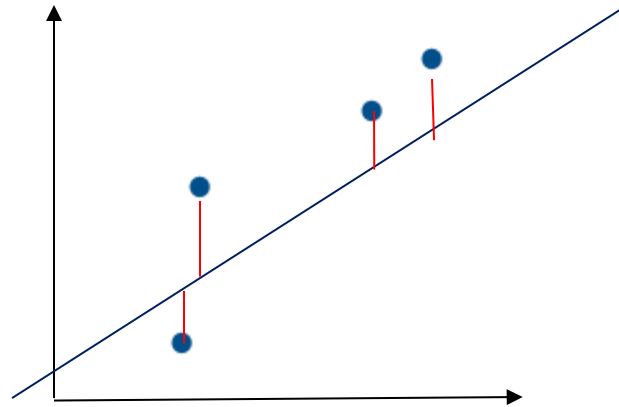
AdaBoost: Updating the weights

- Initialization: $w_i = 1/n$ (n is the number of cases)
- Iteration ($k \rightarrow k+1$):
 - Let e_k = weighted error of C_k
 - $w_i := w_i \cdot e_k / (1 - e_k)$, for correctly classified cases
 - $w_i := w_i$, for misclassified cases
- Next all the weights are normalised (divided by $\sum w_i$).
 - Note that e must satisfy $0 < e < 0.5$
- Voting strategy:
 - The classifier C_k gets weight = $-\log(e_k / (1 - e_k))$

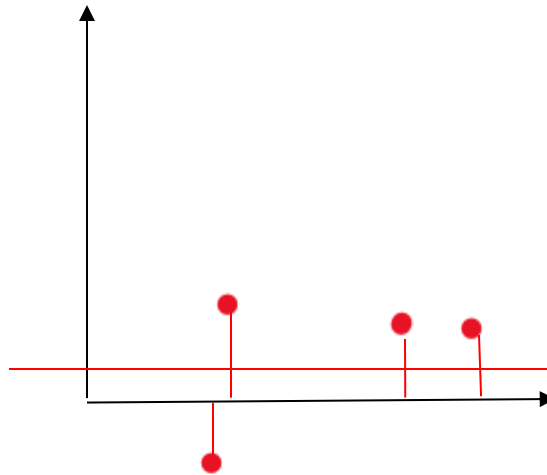
How to boost numeric predictions

- Fit the first model to the data
- Take the residual (actual – predicted)
 - Residual = error on the training set instances
- Train the next model to predict the residuals
 - = train as if target variable was replaced by residuals
- add predictions of second model to those of the first model
- Keep building and adding

1. train some
model $F_0 = h_0$



2. train
another
model h_1 to
predict the
errors



3. combine
models

$$F_1 = F_0 + h_1$$

4. do this for M steps
iteratively

Numeric Boosting properties

- If individual models minimise squared error → boosting minimises the squared error of the ensemble
- **True or false?** The base model should not be linear regression.
 - **True**: sum of lines is still a line, and the first already minimised the squared error
 - **False**: if you use different sets of attributes for different steps

Popular variant: Gradient Boosting

- Developed by Friedman in 1999
- Covers classification and regression
- Assume our dataset has n training pairs (x_i, y_i)
- Now we can use an arbitrary error/loss function (as long as it is differentiable)
 - For example $\frac{1}{2}[y_i - F(x_i)]^2$
- Want to develop a combination of models F and iteratively improve on the error function

Gradient Boosting

- So we start with the best model and try to improve this over and over again

$$F_0(x) = \underset{g}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, g) \quad \text{Constant function}$$

$$F_m(x) = F_{m-1}(x) + \underset{h_m \in H}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + h_m(x_i))$$

- Computationally not feasible for arbitrary loss function

Gradient Boosting

Input: training set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss function $L(y, F(x))$, number of iterations M .

Algorithm:

1. Initialize model with a constant value:

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma).$$

2. For $m = 1$ to M :

1. Compute so-called *pseudo-residuals*:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n.$$

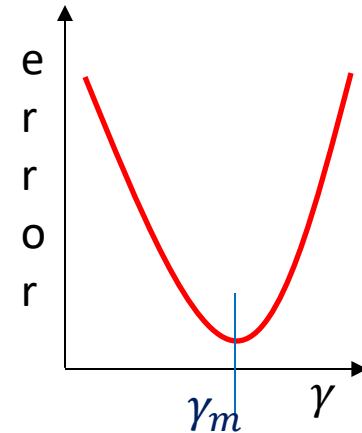
2. Fit a base learner (e.g. tree) $h_m(x)$ to pseudo-residuals, i.e. train it using the training set $\{(x_i, r_{im})\}_{i=1}^n$.
3. Compute multiplier γ_m by solving the following **one-dimensional optimization** problem:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)).$$

4. Update the model:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

3. Output $F_M(x)$.



The loss function

Setting	Loss Function	$-\partial L(y_i, f(x_i))/\partial f(x_i)$
Regression	$\frac{1}{2}[y_i - f(x_i)]^2$	$y_i - f(x_i)$
Regression	$ y_i - f(x_i) $	$\text{sign}[y_i - f(x_i)]$
Regression	Huber	$y_i - f(x_i)$ for $ y_i - f(x_i) \leq \delta_m$ $\delta_m \text{sign}[y_i - f(x_i)]$ for $ y_i - f(x_i) > \delta_m$ where $\delta_m = \alpha\text{th-quantile}\{ y_i - f(x_i) \}$
Classification	Deviance	$k\text{th component: } I(y_i = \mathcal{G}_k) - p_k(x_i)$

To recap

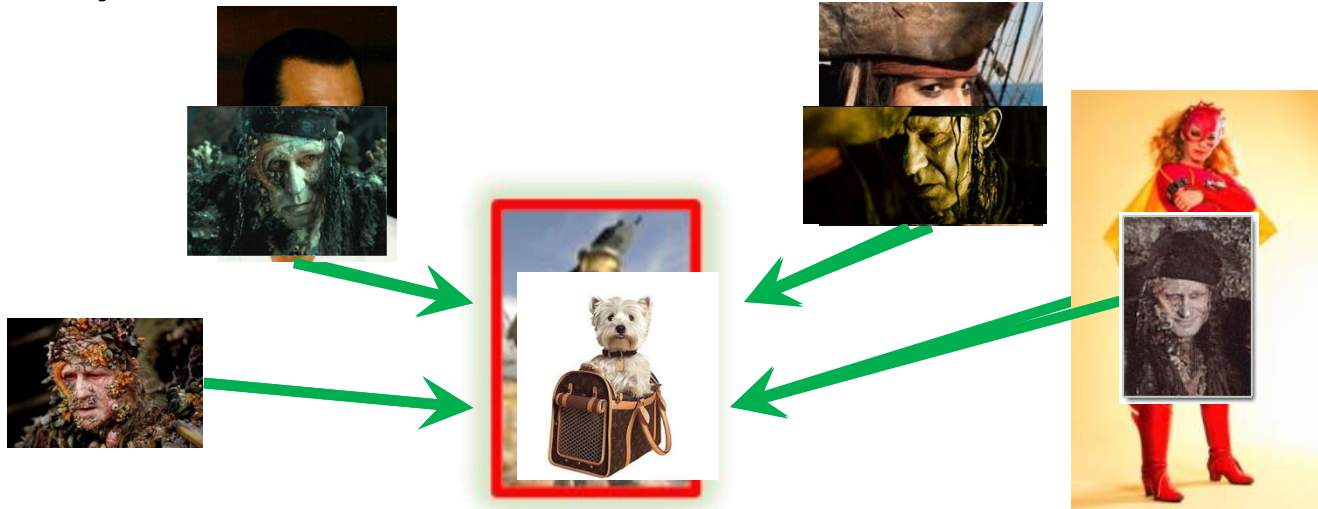
- **Boosting**
 - trains a (weak) classifier
 - Then reweights the instances and trains another on the error, etc.
 - Stops when
 - error is low enough
 - maximum number of classifiers reached
- **AdaBoost**: a particular way of reweighting
- **Gradient Boosting**: more general way with user defined loss/error function

Properties of Boosting

- Reduces the error on the training set exponentially fast
- Most successful with primitive “base classifiers”, e.g., decision stumps, linear regression
- Models are difficult to interpret
- Computational cost

Different models

- So far we had votes and averages
- But they came from the same kind of models



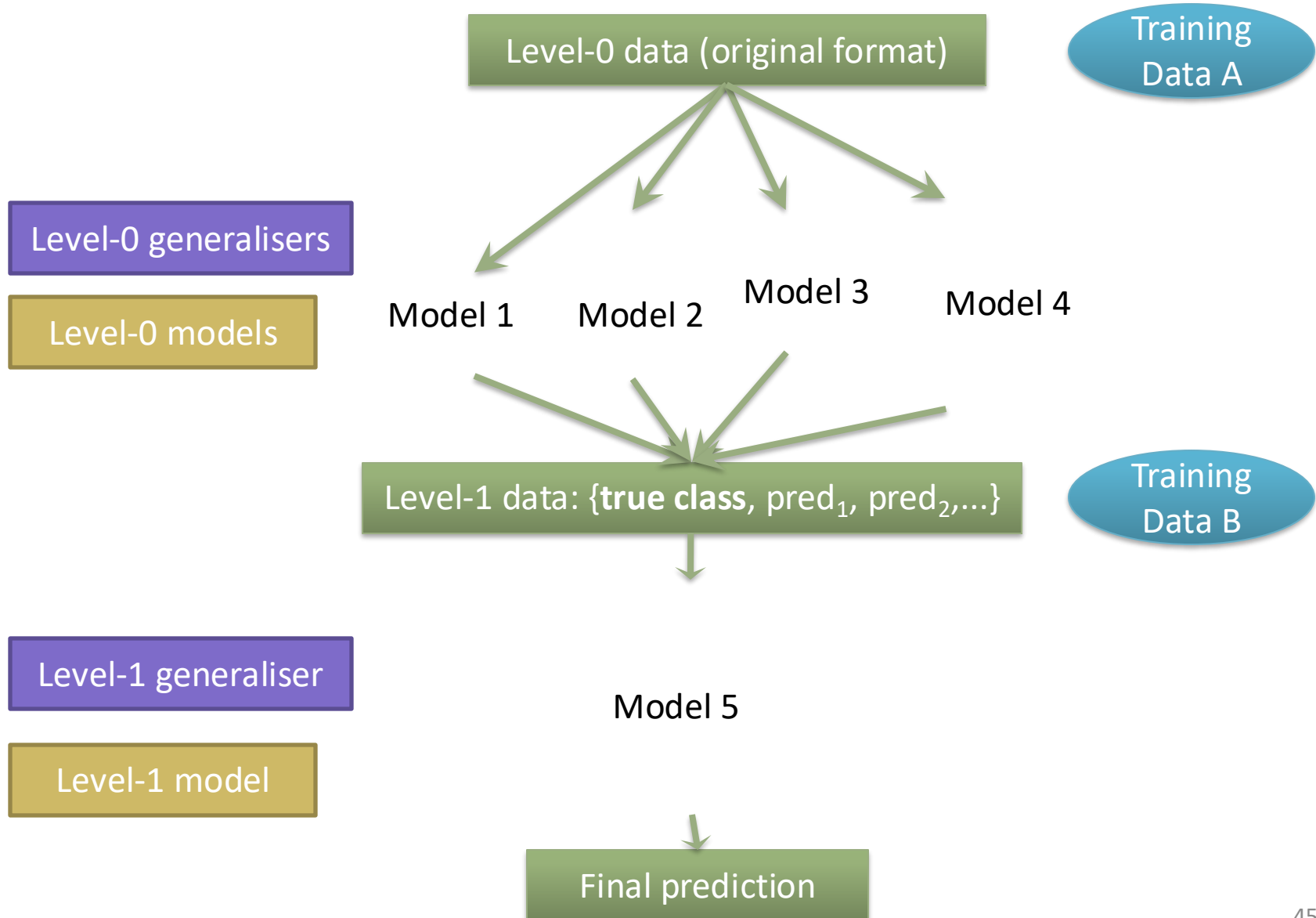
- Can we combine, e.g., a NB with a decision tree and a linear model?



- **Stacking/metalearning**



Stacking classes



Stacking

- Level-1 generaliser can be simple
- Easily adapted for numeric data
- **But:** Not so widely used
 - Difficult to analyse theoretically
 - No generally accepted way of doing the steps



General conclusions

- Unsupervised learning with clustering
- Ensembles usually outperform single models
- Ensembles of “weak” classifiers often outperform single sophisticated classifiers
- Boosting may overfit the data